Nama : Ammar
Nim : 2024573010129
Kelas : TI3B
Matkul : Pemograman Mobile (With Flutter)

1. HTTP: Pengertian dan Cara Memakainya

A. Apa itu HTTP?

HTTP (HyperText Transfer Protocol) adalah protokol komunikasi yang dipakai untuk bertukar data antara **client** (dalam hal ini aplikasi Flutter) dan **server** melalui internet. Cara kerjanya berupa pola **request–response**: client mengirim permintaan (request) ke sebuah alamat, lalu server membalas (response) dengan data.

Pada aplikasi nyata, data tidak ditulis langsung di dalam kode (hardcoded), tetapi diambil dari server melalui API. Aplikasi Flutter tidak bisa mengakses database server secara langsung, sehingga selalu melalui API. Beberapa istilah penting:

- **API:** pintu masuk resmi untuk meminta data ke server, umumnya mengembalikan data berformat JSON.
- **Endpoint:** satu alamat (URL) spesifik pada API, misalnya https://www.themealdb.com/api/json/v1/1/lookup.php?i=52772.
- **Metode HTTP:** GET untuk mengambil data, POST untuk mengirim data baru.

B. Alur kerja HTTP pada aplikasi Flutter

1. Aplikasi (client) mengirim HTTP GET ke endpoint server.
2. Server memproses permintaan dan membalas dengan status code serta isi respons (body) berupa teks JSON.
3. Aplikasi memeriksa status code, mengubah (decode) JSON menjadi objek Dart, lalu menampilkannya di layar.

C. Status code HTTP

Status code menunjukkan hasil dari sebuah permintaan. Aplikasi harus memeriksanya sebelum memproses data.

Status	Arti	Yang dilakukan aplikasi
200	OK	Permintaan berhasil, lanjut memproses data
201	Created	Data berhasil dikirim (khusus POST)
400	Bad Request	Parameter permintaan salah
401 / 403	Unauthorized / Forbidden	Perlu login atau API key
404	Not Found	Endpoint salah atau data tidak ada
500	Server Error	Masalah di server, tampilkan pesan "coba lagi nanti"

D. Cara memakai HTTP di Flutter

Langkah 1 – Tambahkan package http. Package http bukan bagian dari Flutter SDK, sehingga harus ditambahkan lebih dulu lewat terminal:

```
flutter pub add http
```

Perintah tersebut otomatis menambahkan http pada bagian dependencies di file pubspec.yaml.

Langkah 2 – Import package pada file Dart:

```
import 'dart:convert';                    // untuk jsonDecode
import 'package:http/http.dart' as http;  // "as http" = nama panggilan
```

Langkah 3 – Kirim request GET dan periksa status code. Karena butuh waktu, request dipanggil dengan `async` dan `await`:

```
Future<void> ambilData() async {
  final response = await http.get(
    Uri.parse('https://www.themealdb.com/api/json/v1/1/filter.php?c=Seafood'),
  );

  if (response.statusCode == 200) {
    print(response.body);   // isi respons berupa teks (string JSON)
  } else {
    print('Gagal, status: ${response.statusCode}');
  }
}
```

Catatan: response.body masih berupa teks, belum bisa dipakai sebagai data. Kesalahan paling umum adalah tidak memeriksa statusCode lalu langsung memanggil jsonDecode, sehingga muncul FormatException jika server membalas halaman error. Pada Android, pastikan izin INTERNET ada di AndroidManifest.xml agar aplikasi tetap bisa terhubung saat dirilis.

2. JSON: Pengertian dan Cara Memakainya

A. Apa itu JSON?

JSON (JavaScript Object Notation) adalah format teks ringan untuk menyimpan dan mempertukarkan data. JSON mudah dibaca manusia dan mudah diproses mesin, sehingga menjadi format yang paling umum dipakai oleh API. Data JSON disusun dalam pasangan **key : value** dan memiliki dua bentuk utama:

- **Objek** diawali tanda { } , berisi pasangan key–value.
- **Array** diawali tanda [ ] , berisi daftar nilai atau daftar objek.

Contoh JSON berbentuk objek yang di dalamnya terdapat array bernama "meals":

```
{
  "meals": [
    { "strMeal": "Baked salmon with fennel & tomatoes", "idMeal": "52959" },
    { "strMeal": "Cajun spiced fish tacos", "idMeal": "53047" }
  ]
}
```

B. Cara memakai JSON di Flutter

1) Decode JSON dengan jsonDecode(). Fungsi jsonDecode() (dari dart:convert) mengubah teks JSON menjadi struktur Dart. Jenis hasilnya tergantung karakter pertama respons:

Diawali	Bentuk JSON	Cara decode di Dart
[	Array (list)	jsonDecode(body) as List<dynamic>
{	Objek (map)	jsonDecode(body) as Map<String, dynamic>

```
// Respons berbentuk objek { "meals": [ ... ] }
final data = jsonDecode(response.body) as Map<String, dynamic>;
final daftar = data['meals'] as List<dynamic>;   // array ada di dalam key "meals"
print(daftar.first['strMeal']);
```

2) Ubah menjadi objek Dart memakai class model dan factory fromJson. Agar data lebih aman (type safety) dan kode lebih rapi, setiap item JSON diubah menjadi objek dari class model. Field yang bisa bernilai null diberi nilai default memakai `as String? ?? 'default'`.

```
class Meal {
  final String id;
  final String nama;
  final String gambar;

  const Meal({required this.id, required this.nama, required this.gambar});

  factory Meal.fromJson(Map<String, dynamic> json) {
    return Meal(
      id: json['idMeal'] as String? ?? '',
      nama: json['strMeal'] as String? ?? 'Tanpa Nama',
      gambar: json['strMealThumb'] as String? ?? '',
    );
  }
}
```

3) Ubah seluruh array menjadi List<Meal>:

```
final hasil = daftar
    .map((item) => Meal.fromJson(item as Map<String, dynamic>))
    .toList();
```

Catatan: salah memilih tipe akan menimbulkan error "type 'List<dynamic>' is not a subtype of type 'Map<String, dynamic>'". Tanpa nilai default, field yang null juga akan menimbulkan error "type 'Null' is not a subtype of type 'String'".

3. FutureBuilder pada Flutter: Pengertian dan Cara Memakainya

A. Konsep dasar: Future dan async/await

Mengambil data dari internet membutuhkan waktu. Dart menangani proses seperti ini dengan **Future**, yaitu objek yang berarti "nanti akan ada nilai". Kata kunci `async` menandai fungsi yang memiliki proses menunggu, sedangkan `await` menunggu Future selesai tanpa membekukan layar. Analoginya seperti memesan makanan di restoran: kita menerima nomor pesanan (Future) dan menunggu sampai dipanggil (await).

B. Apa itu FutureBuilder?

FutureBuilder adalah widget Flutter yang membangun tampilan secara otomatis berdasarkan keadaan sebuah Future. Dengan FutureBuilder, kita tidak perlu membuat sendiri variabel loading dan error, karena Flutter menyediakan objek **snapshot** yang mewakili keadaan Future tersebut. Aplikasi yang mengambil data dari internet selalu berada pada salah satu dari tiga keadaan:

Keadaan	Cara memeriksa di snapshot	Yang dilihat pengguna
Menunggu	snapshot.connectionState == ConnectionState.waiting	CircularProgressIndicator
Gagal	snapshot.hasError	Pesan error + tombol "Coba Lagi"
Berhasil	snapshot.data	Daftar data (ListView.builder)

C. Cara memakai FutureBuilder

1. **Buat fungsi async** yang mengembalikan Future, misalnya fungsi yang memanggil http.get lalu mengubah JSON menjadi List<Meal>.
2. **Simpan Future di variabel** pada State dan isi di initState(), **bukan** dibuat di dalam build().
3. **Pasang FutureBuilder** dan tangani tiga keadaan di dalam builder.

```
class _HalamanKatalogState extends State<HalamanKatalog> {
   late Future<List<Meal>> futureMeal;

   @override
   void initState() {
     super.initState();
     futureMeal = ambilDaftarMeal();   // permintaan dikirim sekali saja
   }

   void muatUlang() {
     setState(() { futureMeal = ambilDaftarMeal(); });
   }

   @override
   Widget build(BuildContext context) {
     return Scaffold(
       body: FutureBuilder<List<Meal>>(
         future: futureMeal,
         builder: (context, snapshot) {
           // 1. Menunggu
           if (snapshot.connectionState == ConnectionState.waiting) {
             return const Center(child: CircularProgressIndicator());
           }
           // 2. Gagal
           if (snapshot.hasError) {
             return Center(child: Text('Gagal: ${snapshot.error}'));
           }
           // 3. Berhasil
           final daftar = snapshot.data ?? [];
           return ListView.builder(
             itemCount: daftar.length,
             itemBuilder: (context, index) => ListTile(
               title: Text(daftar[index].nama),
               subtitle: Text(daftar[index].area),
             ),
           );
         },
       ),
     );
   }
 }
```

Penting: jangan menulis future: ambilDaftarMeal() langsung di dalam build(). Fungsi build() bisa dipanggil berkali-kali (misalnya saat layar digulir), sehingga permintaan HTTP akan terkirim terus-menerus tanpa henti. Simpan Future di variabel dan isi sekali di initState().

D. Perbandingan setState manual dan FutureBuilder

Aspek	setState manual	FutureBuilder
Kode	Lebih panjang	Lebih ringkas
Variabel keadaan	sedangMemuat, pesanError, dan data (3 variabel)	Cukup satu variabel Future
Loading dan error	Dikelola sendiri	Disediakan oleh snapshot
Cocok untuk	Aksi tombol, proses bertahap	Menampilkan hasil satu permintaan

Kesimpulan

HTTP adalah protokol untuk meminta data dari server melalui internet, JSON adalah format data yang dikirim server sebagai balasan, dan FutureBuilder adalah widget yang membantu menampilkan data tersebut sesuai keadaan loading, gagal, atau berhasil. Tiga hal yang selalu diingat: periksa statusCode, tangani keadaan gagal, dan jangan membuat Future di dalam build().